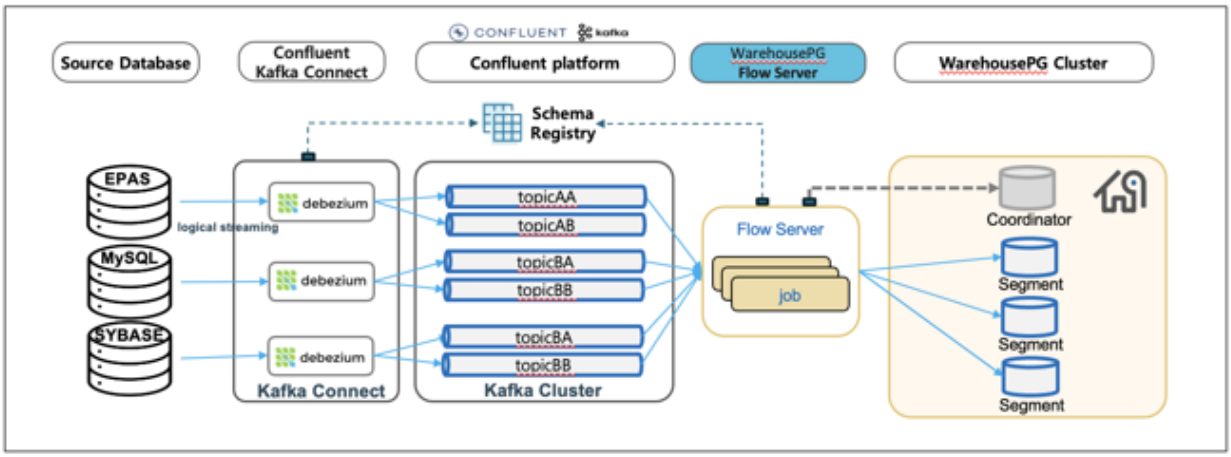
Flowserver 설치/운영 가이드

Overall CDC Architecture



1.Setup WarehousePG (Target DB)

1-1. WarehousePG 모든 노드에 Flowserver 설치

```
$ sudo dnf install whpg7-flow-server*
```

1-2. Flowserver Node 에 Flowserver RPM 과 WarehousePG RPM 설치

```
$ sudo dnf install warehouse-pg-7.2.1_WHPG-1*
$ sudo dnf install whpg7-flow-server*
```

1-3. WarehousePG 에 flowserver 를 위한 Extension 생성

```
$ psql -d whpg
create extension fs_formatter;
```

참고. 토픽에서 테이블로 로딩할 때 오류가 발생하고 정상적인 로딩이 이루어지지 않는 경우를 확인하는 SQL

```
SELECT cmdtime, errmsg, encode(rawbytes, 'escape') AS rawbytes
FROM gp_read_persistent_error_log('"public"."whpgfs_ext_avro"')
WHERE cmdtime >= '2025-09-15T12:18:02.009596Z' ORDER BY cmdtime DESC;
```

2. Flow Server Node 에 Flowserver 구성.

2-1. flowserver 설정 파일 생성

아래는 gpadmin user 에서 수행해야 함. Gpadmin user 가 없을 시는 생성 필요.

아래에서 port 는 수정 가능. Default port 는 6060 임.

```
sudo tee -a gpconfigs/flowcfg.json << EOF
{
    "Host": "",
    "Port": 6060,
    "Gpfdist": {
        "Host": "10.0.1.254",   # Flow Server IP
        "Port": 6070,
        "ReuseTables": false    #
    },
    "Prometheus": {
        "Host": "",
        "Port": 9080,
        "MetricsPath": "/flow_metrics"
    },
    "DebugPort": 6080,
    "Logging": {
        "SplitLogByJob": false,
        "FrontendLevel": "debug",
        "BackendLevel": "info"
    }
}
EOF
```

2-2. flowserver 를 linux 서비스로 등록

```
sudo tee -a /etc/systemd/system/flowserver.service << EOF
[Unit]
Description=WHPG Flow Server

[Service]
Type=simple
User=gpadmin
Group=gpadmin
ExecStart=/usr/local/greenplum-db-7.2.1-WHPG/bin/flowserver -c
/home/gpadmin/gpconfigs/flowservercfg.json -l /home/gpadmin/gpAdminLogs
Restart=on-abnormal

[Install]
WantedBy=multi-user.target
EOF
```

Flow Server 시작 / 확인

```
sudo systemctl daemon-reload
sudo systemctl start flowserver
sudo systemctl status flowserver
```

Flow Server 로그 확인.

```
tail -f ./gpAdminLogs/whpg_flow_server-2025-09-09.log
```

3. Flowserver 에 job 등록

Job 은 Confluent(Kafka) topic 당 하나씩 생성해야 합니다.

```
sudo tee -a ~/topic_name.yaml << EOF
version: v1.0
source:
  kafka:
    brokers:
      - localhost:9092
    topic: epas17.public.avro
    value:
      avro:
        column:
          name: jdata
          type: JSON
        schema_registry: localhost:8081
    control:
      consistency: exactly-once
      fallback_offset: earliest
    task:
      batch_size:
        interval_ms: 100
        max_count: 10
      window_size: 0
      #window_statement: <udf_or_sql_to_run>
      #prepare_statement: <udf_or_sql_to_run>
      #teardown_statement: <udf_or_sql_to_run>
target:
  database:
    host: 10.0.27.155
    port: 5432
    user: gpadmin
    password: gpadmin
    dbname: whpg
    #staging_schema: public
    error_limit: "5"
    tables:
    - table: avro
      schema: public
      mode:
        # one of the below insert, update, or merge modes
        insert: {}
        #update:
        #  match_columns: id
        #  update_columns: name, month, amount_paid
        #  update_condition: id = id
      mapping:
        id: ((((jdata->'after')::json->'epas17.public.avro.Value')::json->'id')::json-
>>'long')::int8
        name: ((((jdata->'after')::json->'epas17.public.avro.Value')::json-
>'name')::json->>'string')::varchar(100)
        month: ((((jdata->'after')::json->'epas17.public.avro.Value')::json-
>'month')::json->>'int')::int4
        amount_paid: ((((jdata->'after')::json->'epas17.public.avro.Value')::json-
>'amount_paid')::json->>'long')::int8
option:
  schedule:
    running_duration: 2s
    auto_stop_restart_interval: 2s
    max_restart_times: 1
```

```
EOF
```

Job 등록
```
flowcli submit --name topic_name ~/topic_name.yaml
```

Job 폴더에 있는 모든 job 동시 등록
```
flowcli submit ~/flowserver/
```

Job 실행 시작
```
flowcli start topic_name
```

Job 실행 시작. kafka topic 처음부터 다시 적제.
```
flowcli start topic_name --reset-to-earliest
```

Job 중지
```
flowcli stop topic_name
```

Job 삭제
```
flowcli remove topic_name
```

Job 확인
```
flowcli list
```

참고 1. Flowserver Configuration

WareHousePG flow server configuration file.

Synopsis

```
{
    "Host": "",
    "Port": 6060,
    "Gpfdist": {
        "Host": "",
        "Port": 6070,
        "ReuseTables": true
    },
    "Prometheus": {
        "Host": "",
        "Port": 9080,
        "MetricsPath": "/flow_metrics"
    },
    "DebugPort": 6080,
    "Logging": {
        "SplitLogByJob": false,
        "FrontendLevel": "debug",
        "BackendLevel": "info"
    }
}
```

Description

This file contains the configuration for the WareHousePG flow server. It includes settings for the server host and port, gpfdist settings, Prometheus metrics, debug port, and logging options. If the file is not present, the server will use default values.

Keywords and Values

- Host: The hostname or IP address of the server. The default is an empty string, which means it will listen on all interfaces.
- Port: The port number on which the server listens for incoming connections. The default is 6060.
- Gpfdist: Configuration options for the gpfdist service.
    - Host: The hostname or IP address of the gpfdist service. The default is an empty string, which means it will listen on all interfaces.
    - Port: The port number on which the gpfdist service listens. The default is 6070.
    - ReuseTables: Whether to reuse existing tables in the database. The default is false, not reuse the external table. When you reuse external tables, flow server generates the external table name using a hash of various load configuration property values. By default, flow server drops the external table associated with a load operation (if one exists) and creates a new external table when you (re)start the job. If you do not reuse external tables, the external table name is based on the job name.
- Prometheus: Configuration options for the Prometheus metrics endpoint.
    - Host: The hostname or IP address of the Prometheus service. The default is an empty string, which means it will listen on all interfaces.
    - Port: The port number on which the Prometheus service listens.
    - MetricsPath: The path to the metrics endpoint.
- DebugPort: The port number for the debug server.
- Logging: Configuration options for logging. The valid log levels are debug, info, warn, error and fatal.
    - SplitLogByJob: Whether to split logs by job. The default is true, meaning logs will be separated by job.
    - FrontendLevel: The logging level for the frontend/stdout. The default is "info".
    - BackendLevel: The logging level for the backend/log file. The default is "debug".

참고 2. flowserver CLI 사용법

The instance of the WareHousePG flow server.

Synopsis

```
flowserver [-c | --config <config_file>]
[-l | --log-dir <log_directory>]
```

Description

The WareHousePG flow server is a streaming data processing server that supports various data sources. It can be configured to read from sources like Kafka or RabbitMQ and write to a WareHousePG database. The server can be started with a configuration file that specifies the server settings, data sources, and target database configurations.

The server supports different data formats such as JSON, Avro, and CSV, and allows for flexible task management and scheduling options.

Options

- -c, --config <config_file>: Specifies the path to the configuration file for the flow server. If not provided, the server will use default settings.
- -l, --log-dir <log_directory>: Specifies the directory where the server logs will be stored. If not provided, logs will be written to the default log directory which is $HOME/gpAdminLogs.

참고 3. Flowserver Job Configuration

WareHousePG flow server job YAML configuration file for Kafka or RabbitMQ source.

Synopsis

Kafka source configuration example:

```
version: v1.0
source:
  kafka:
    brokers:
      - <kafka_broker_host:broker_port>
    topic: <kafka_topic>
    value:
      <data_format>:
        <column_spec>
        <other_props>
    control:
      consistency: at-most-once | at-least-once | exactly-once | none
      fallback_offset: earliest | latest
    task:
      batch_size:
        interval_ms: <wait_time>
        max_count: <number_of_rows>
      window_size: <num_batches>
      window_statement: <udf_or_sql_to_run>
      prepare_statement: <udf_or_sql_to_run>
      teardown_statement: <udf_or_sql_to_run>
    properties:
      <source_property_name>: <source_property_value>

RabbitMQ source configuration example:

version: v1.0
source:
  rabbitmq:
    server: <rmq_user>:<rmq_password>@<rmq_host>:<rmq_port>
    vhost: <vhost_name>
    queue: <queue_name>
    format:
      <data_format>:
        <column_spec>
        <other_props>
    control:
      consistency: at-most-once | at-least-once
    task:
      batch_size:
        interval_ms: <wait_time>
        max_count: <number_of_rows>
      window_size: <num_batches>
      window_statement: <udf_or_sql_to_run>
      prepare_statement: <udf_or_sql_to_run>
```

```
      teardown_statement: <udf_or_sql_to_run>
    properties:
      <source_property_name>: <source_property_value>
```

Target configuration example:

```
target:
  database:
    host: <whpg_host>
    port: <whpg_port>
    user: <user_name>
    password: <password>
    dbname: <dbname>
    staging_schema: <staging_schema>
    error_limit: <num_errors> | <percentage_errors>
    filter_expression: <filter_string>
    tables:
    - table: <table_name>
      schema: <schema_name>
      mode:
        # one of the below insert, update, or merge modes
        insert: {}
        update:
          <mode_specific_property>: <value>
          ...
        merge:
          <mode_specific_property>: <value>
          ...
      mapping:
        <target_column_name> : <source_column_name> | <expression>
      filter: <output_filter_string>
option:
  schedule:
    running_duration: <run_time>
    auto_stop_restart_interval: <restart_time>
    max_restart_times: <num_restarts>
```

Where the mode that you can specify for update and merge mode follow:

```
update:
  match_columns: [<match_column_names>]
  order_columns: [<order_column_names>]
  update_columns: [<update_column_names>]
  update_condition: <update_condition>
merge:
  match_columns: [<match_column_names>]
  update_columns: [<update_column_names>]
  order_columns: [<order_column_names>]
  update_condition: <update_condition>
  delete_condition: <delete_condition>
```

Where data_format, column_spec, and other_props are one of the following blocks:

```
json:
  column:
    name: <column_name>
    type: json | jsonb
avro:
  column:
    name: <column_name>
```

```
    type: json
    schema_registry: <schema_registry_url> | schema_file: <schema_file_path>
csv:
  columns:
    - name: <column_name>
      type: <column_data_type>
    ...
  delimiter: <delim_char>
  quote: <quote_char>
  null_string: <nullstr_val>
  escape: <escape_char>
  newline: <newline_val>
  force_not_null: <columns>
  force_quote: <columns>
  fill_missing_fields: <boolean>
```

Description

This file contains the configuration for the WareHousePG flow server job that uses Kafka or RabbitMQ as a source. It includes settings for the source connection, data format, task management, and target database configuration. The job can be scheduled to run for a specific duration and can automatically stop and restart based on the defined intervals.

Keywords and Values

- version: The version of the configuration format.
- source: The source configuration for the job.
    - kafka: Configuration for Kafka source.
        - brokers: List of Kafka broker addresses.
        - topic: The Kafka topic to consume messages from.
        - value: Configuration for the value data format of the Kafka messages.
            - <data_format>: The format of the data (e.g., json, avro, csv).
                - <column_spec>: Specification of the columns in the data.
                - <other_props>: Other properties specific to the data format.
        - control: Control settings for the Kafka source.
            - consistency: The consistency level for message processing (e.g., at-most-once). The valid values are at-most-once, at-least-once, exactly-once and none. The default is exactly-once which means the job will store the offsets in the database and will not reprocess messages that have already been processed. at-most-once means the job will not store the offsets in the database and commit the offset to Kafka before committing the data loading transaction in database. at-least-once means the job will not store the offsets in the database and commit the offset to Kafka after committing the data loading transaction in database. none means the job will not store the offsets in the database and will auto-ack the offset.
            - fallback_offset: The offset to use if there is offset gap between the record in history store and Kafka watermark. The valid values are earliest and latest. If the value is not specified, An error will be raised if there is offset gap.
        - task: Task management settings.
            - batch_size: Controls the batch size for processing messages.
                - interval_ms: The time interval to wait for new messages before committing loading transactions. The default is 1000 milliseconds.
                - max_count: The maximum number of rows to process in a batch. The default is 0, which means flow server ignores this setting. If both interval_ms and max_count are set, the job will wait for

the specified time interval or until the maximum number of rows is reached, whichever comes first.
- window_size: The number of batches to process before running window_statement.
- window_statement: SQL or UDF to run for each window_size of batch data.
- prepare_statement: SQL or UDF to run before starting the job.
- teardown_statement: SQL or UDF to run after the job stops.

- properties: Additional properties for the source.
  - <source_property_name>: The name of the property.
  - <source_property_value>: The value of the property.

- rabbitmq: Configuration for RabbitMQ source.
  - server: The RabbitMQ server connection string.
  - vhost: The RabbitMQ virtual host to use.
  - queue: The RabbitMQ queue to consume messages from.
  - format: Configuration for the data format of the RabbitMQ messages.
    - <data_format>: The format of the data (e.g., json, csv).
      - <column_spec>: Specification of the columns in the data.
      - <other_props>: Other properties specific to the data format.
  - control: Control settings for the RabbitMQ source.
    - consistency: The consistency level for message processing (e.g., at-most-once). The valid values are at-most-once and at-least-once. The default is at-least-once which means the job will store the offsets in the database and will not reprocess messages that have already been processed. at-most-once means the job will not store the offsets in the database and commit the offset to RabbitMQ before committing the data loading transaction in database.
  - task: It is same as Kafka source task configuration.
  - properties: Additional properties for the source.
    - <source_property_name>: The name of the property.
    - <source_property_value>: The value of the property.

- target: The target database configuration for the job.
  - database: Configuration for the target database.
    - host: The hostname or IP address of the WareHousePG server.
    - port: The port number of the WareHousePG server.
    - user: The username to connect to the WareHousePG server.
    - password: The password for the user.
    - dbname: The name of the database to connect to.
    - staging_schema: The schema to use for creating internal tables. The default is public
    - error_limit: The limit for errors before stopping the job, can be a number or a percentage.
    - filter_expression: The filter to apply to the input data before loading data into the target tables. A valid filter expression is a SQL WHERE clause without the WHERE keyword.
    - tables: List of tables to write data to.
      - table: The name of the target table.
      - schema: The schema of the target table.
      - mode: The mode for writing data to the table.
        - insert: Insert mode.
        - update: Update mode with specific properties.
        - merge: Merge mode with specific properties.
      - mapping: Column mapping between source and target.
        - <target_column_name>: The name of the target column.

            - <source_column_name>: The name of the source column or an expression to compute the value.
        - filter: A filter expression to apply to the target table. A valid filter expression is a SQL WHERE clause without the WHERE keyword and may reference one or more <source_column_name>.
- option: Additional options for the job.
    - schedule: Scheduling options for the job.
        - running_duration: The duration for which the job should run.
        - auto_stop_restart_interval: The interval after which the job should automatically stop and restart.
        - max_restart_times: The maximum number of times the job can be restarted automatically.

참고 4. flowcli CLI 사용법

Command line interface for WareHousePG flow server.

Synopsis

```
flowcli [subcommand] [flags]
[--host <host>]
[--port <port>]
[--log-dir <log_directory>]
```

Description

The WareHousePG flowcli is a command line interface for managing the WareHousePG streaming server jobs. It allows users to submit, start, stop, and monitor jobs.

Subcommands

- submit: Submit a configuration file or directory containing YAML files to the server for job submission. If a directory is provided, all YAML files in that directory will be submitted as separate jobs.

  Flags

    - -n | --name <job_name>: Specify a name for the job. If not provided, the job name will be derived from the configuration file name.
    - -s | --start: Immediately start the job after submission. If specified, you also can add the flags of start subcommand.
- start: Start a job or multiple jobs by their names.

  Flags

    - --all: Start all jobs that are not in running state.
    - --stop-at-eof: Stop the job when the end of the input data is reached. This is applicable for jobs that read from a source like Kafka or RabbitMQ.
    - --reset-to-earliest: Reset to the earliest offset in the source on job starting. This is applicable for jobs that read from Kafka or RabbitMQ stream.
    - --reset-to-latest: Reset to the latest offset in the source on job starting. This is applicable for jobs that read from Kafka or RabbitMQ stream.
    - --reset-to-timestamp <timestamp>: Reset to the specified timestamp in the source on job starting. This is applicable for jobs that read from Kafka or RabbitMQ stream.

    - --skip-explain: Skip the SQL explain phase of the job on starting. This is helpful for reducing the startup time of the job.
- stop: Stop a job or multiple running jobs by their names.

  Flags

    - --all: Stop all running jobs.
- remove: Remove a job or multiple jobs by their names.

  Flags

    - --all: Remove all jobs which are not in running state.
- list: List all jobs with their status and details.

  Flags

    - -n | --name-only: Only show the names of the jobs without details.
- monitor: Monitor the progress of a job or multiple jobs by their names.

Flags

- --host <host>: The hostname or IP address of the WareHousePG flow server. Default is localhost.
- --port <port>: The port number on which the WareHousePG flow server is running. Default is 6060.
- --log-dir <log_directory>: The directory where the logs will be stored. Default is $HOME/gpAdminLogs.

참고 5. Confluent(kafka)의 topic 확인하는 예시.

Topic 리스트 확인
```
kafka-topics --bootstrap-server localhost:9092 --list
```

특정 Topic 구성 확인
```
kafka-topics --bootstrap-server localhost:9092 --describe --topic epas17.public.avro
```

특정 Topic 삭제
```
kafka-topics --bootstrap-server localhost:9092 --delete --topic epas17.public.avro
```

특정 Topic Consumer 실행으로 데이터 확인 (CSV, JSON)
```
kafka-console-consumer --bootstrap-server localhost:9092 --topic
epas17.public.customer --from-beginning
```

Topic Consumer 실행으로 특정 topic 데이터 확인 (AVRO)
```
sudo kafka-avro-console-consumer --bootstrap-server localhost:9092 --topic
epas17.public.avro_from_kafka --from-beginning --property
schema.registry.url=http://localhost:8081
```

Topic Consumer 실행으로 특정 topic 의 특정 데이터(offset) 확인 (AVRO)

```
sudo kafka-avro-console-consumer --bootstrap-server localhost:9092 --topic
tpcc.public.bmsql_warehouse --partition 0 --offset 7 --max-messages 1
```

특정 Topic 건수/Offset 확인

```
kafka-get-offsets --bootstrap-server localhost:9092 --topic epas17.public.customer
```

참고 6. Flowserver job 을 자동으로 생성하는 스크립트

파일명: generate_logic.awk

```
# Debezium JSON 파싱 로직 생성 스크립트 (Nullable/Non-Nullable 모두 처리)
# 사용법: awk -v server_name="tpcc" -f generate_logic.awk schema.txt

# PostgreSQL 타입을 Avro/JSON 키(int, long 등)로 변환하는 함수
function get_avro_key(pg_type) {
    if (pg_type ~ /integer/) return "int";
    if (pg_type ~ /bigint/) return "long";
    if (pg_type ~ /timestamp/) return "long";
    if (pg_type ~ /numeric/) return "double";
    if (pg_type ~ /character varying|varchar|text/) return "string";
    return "string"; # 기본값
}

# 스크립트 시작 시 필드 구분자를 | 로 설정
BEGIN { FS = "|" }

# 첫 번째 줄에서 테이블/스키마 이름을 추출하여 JSON 경로 생성
NR == 1 {
    match($0, /"([^"]+)"/);
    full_table_name = substr($0, RSTART + 1, RLENGTH - 2);
    json_path = sprintf("->\x27%s.%s.Value\x27", server_name, full_table_name);
}

# 컬럼 정의 라인만 처리
NR > 2 && NF > 1 && !/---/ {
    # 컬럼명과 타입 추출 및 공백 제거
    column = $1;
    gsub(/^[[:space:]]+|[[:space:]]+$/, "", column);

    pg_type = $2;
    gsub(/^[[:space:]]+|[[:space:]]+$/, "", pg_type);

    if (pg_type == "") next;

    # 최종 SQL CAST 타입 설정
    sql_cast_type = pg_type;
    sub("character varying", "varchar", sql_cast_type);
    if (sql_cast_type == "integer") sql_cast_type = "int8";

    # Avro/JSON 키 가져오기
    avro_key = get_avro_key(pg_type);
```

```
    # 'before'와 'after' 블록의 데이터 컨테이너 경로
    before_container = sprintf("(((jdata->\x27before\x27)::json%s)::json)",
json_path);
    after_container = sprintf("(((jdata->\x27after\x27)::json%s)::json)", json_path);

    # 1. Nullable 형식 경로, 2. Non-Nullable 형식 경로
    nested_before = sprintf("(%s->\x27%s\x27)::json->>\x27%s\x27", before_container,
column, avro_key);
    direct_before = sprintf("(%s->>\x27%s\x27)", before_container, column);

    nested_after = sprintf("(%s->\x27%s\x27)::json->>\x27%s\x27", after_container,
column, avro_key);
    direct_after = sprintf("(%s->>\x27%s\x27)", after_container, column);

    # COALESCE로 두 경로를 합침
    coalesced_before = sprintf("COALESCE(%s, %s)", nested_before, direct_before);
    coalesced_after = sprintf("COALESCE(%s, %s)", nested_after, direct_after);

    # timestamp 타입 특별 처리
    if (pg_type ~ /timestamp/) {
        final_before = sprintf("TO_TIMESTAMP((%s)::bigint / 1000000.0)::timestamp",
coalesced_before);
        final_after = sprintf("TO_TIMESTAMP((%s)::bigint / 1000000.0)::timestamp",
coalesced_after);
    } else {
        final_before = sprintf("(%s)::%s", coalesced_before, sql_cast_type);
        final_after = sprintf("(%s)::%s", coalesced_after, sql_cast_type);
    }

    # 최종 CASE 구문 출력
    printf "        %s: CASE WHEN (jdata->>\x27op\x27)=\x27d\x27 THEN %s ELSE %s
END\n", column, final_before, final_after;
}
```

파일명: makeyaml.sh

```
#!/bin/bash

ghost="whpg-m"
gport="5432"
user="gpadmin"
password="gpadmin"
DB="whpg"
schema="public"
Dir_Yml="/home/gpadmin/yaml/"
broker="10.0.1.254:9092"
registry="10.0.1.254:8081"
topic="tpcc.public.bmsql_history"

psql -d "$DB" -h "$ghost" -At -c "SELECT table_name FROM information_schema.tables
WHERE table_schema='$schema' AND table_type='BASE TABLE' AND table_name not like
'whpgfs_hs_%';" > schemalist.txt

if [ `cat schemalist.txt | wc -l` -eq 0 ]
then
echo "Table doesn't exist. Please enter correct name"
echo "Script exiting"
exit;
```

```
fi

for i in `cat schemalist.txt`
do
echo "version: v1.0" > $Dir_Yml$i.yml
echo "source:" >> $Dir_Yml$i.yml
echo "  kafka:" >> $Dir_Yml$i.yml
echo "    brokers:" >> $Dir_Yml$i.yml
echo "      - $broker" >> $Dir_Yml$i.yml
echo "    topic: tpcc.public.$i " >> $Dir_Yml$i.yml
echo "    value:" >> $Dir_Yml$i.yml
echo "      avro:" >> $Dir_Yml$i.yml
echo "        column:" >> $Dir_Yml$i.yml
echo "          name: jdata" >> $Dir_Yml$i.yml
echo "          type: json" >> $Dir_Yml$i.yml
echo "        schema_registry: $registry" >> $Dir_Yml$i.yml
echo "    control:" >> $Dir_Yml$i.yml
echo "      consistency: exactly-once" >> $Dir_Yml$i.yml
echo "      fallback_offset: earliest" >> $Dir_Yml$i.yml
echo "    task:" >> $Dir_Yml$i.yml
echo "      batch_size:" >> $Dir_Yml$i.yml
echo "        interval_ms: 100" >> $Dir_Yml$i.yml
echo "        max_count: 1000" >> $Dir_Yml$i.yml
echo "      window_size: 0" >> $Dir_Yml$i.yml
echo "target:" >> $Dir_Yml$i.yml
echo "  database:" >> $Dir_Yml$i.yml
echo "    host: $ghost" >> $Dir_Yml$i.yml
echo "    port: $gport" >> $Dir_Yml$i.yml
echo "    user: $user" >> $Dir_Yml$i.yml
echo "    password: $password" >> $Dir_Yml$i.yml
echo "    dbname: $DB" >> $Dir_Yml$i.yml
echo "    #staging_schema: public" >> $Dir_Yml$i.yml
echo "    error_limit: \"5\"" >> $Dir_Yml$i.yml
echo "    tables:" >> $Dir_Yml$i.yml
echo "    - table: $i" >> $Dir_Yml$i.yml
echo "      schema: $schema" >> $Dir_Yml$i.yml
echo "      mode:" >> $Dir_Yml$i.yml
echo "        merge:" >> $Dir_Yml$i.yml
echo "          match_columns:" >> $Dir_Yml$i.yml
# awk 스크립트를 AWK_SCRIPT 변수에 저장
AWK_SCRIPT='
/^Distributed by/ {
    gsub(/.*\(|\).*/, "")
    split($0, a, ", ")
    for(i in a) {
        print "          - " a[i]
    }
}'

# 변수를 사용하여 awk 명령어 실행
psql -d "$DB" -h "$ghost" -c "\d $gschema.$i" | awk "$AWK_SCRIPT" >> "$Dir_Yml$i.yml"

echo "          delete_condition: (jdata->>'op')::text = 'd'" >> $Dir_Yml$i.yml
echo "      mapping:" >> $Dir_Yml$i.yml

psql -d $DB -h "$ghost" -c "\d $gschema.$i" > schema.txt
awk -v server_name="tpcc" -f generate_logic.awk schema.txt >> $Dir_Yml$i.yml
done
```

참고 7. Job configuration file for Merge Mode (Insert, Update, Delete)

```yaml
version: v1.0
source:
  kafka:
    brokers:
      - 10.0.1.254:9092
    topic: epas17.public.avro
    value:
      avro:
        column:
          name: jdata
          type: JSONB
        schema_registry: 10.0.1.254:8081
    control:
      consistency: exactly-once
      fallback_offset: earliest
    task:
      batch_size:
        interval_ms: 1000
        max_count: 100
      window_size: 0
      #window_statement: <udf_or_sql_to_run>
      #prepare_statement: <udf_or_sql_to_run>
      #teardown_statement: <udf_or_sql_to_run>
target:
  database:
    host: 10.0.27.155
    port: 5432
    user: gpadmin
    password: gpadmin
    dbname: whpg
    #staging_schema: public
    error_limit: "5"
    tables:
    - table: avro
      schema: public
      mode:
        merge:
          match_columns:
            - id
          #  - name
          #  - month
          #  - amount_paid
          #update_columns: [ name, month, amount_paid ]
          #update_condition: (jdata->>'op')::text = 'u'
          delete_condition: (jdata->>'op')::text = 'd'
      mapping:
        id: fs_udf(jdata, 'id')::int8
        name: fs_udf(jdata, 'name')::varchar(100)
        month: fs_udf(jdata, 'month')::int4
        amount_paid: fs_udf(jdata, 'amount_paid')::int8
option:
  schedule:
    running_duration: 2s
    auto_stop_restart_interval: 2s
    max_restart_times: 1
```